Université Abdelmalek Essaâdi
ENSA Al Hoceima

Compte Rendu : TP N°5

Java SE (JSE) - ID1

Réalisé par :
Hidaya El hichami

Présenté à :
Pr. Mohamed CHERRADI

Table des matières

1 Introduction **2**

2 Partie 1 : Exceptions système **2**
2.1 Exercice 1 : Division sécurisée 2
2.2 Exercice 2 : NullPointerException 3
2.3 Exercice 3 : Tableau sécurisé 4
2.4 Exercice 4 : Conversion d'entier 5
2.5 Exercice 5 : Choix d'exception 6
2.6 Exercice 6 : IllegalStateException 7
2.7 Exercice 7 : Propagation 8
2.8 Exercice 8 : Checked Exception 8
2.9 Exercice 10 : Comparaison 9

3 Partie 2 : Exceptions personnalisées **10**
3.1 Exercice 1 : Compte bancaire 10
3.2 Exercice 2 : Montant invalide 11
3.3 Exercice 3 :Double validation 12
3.4 Exercice 4 : Inscription Utilisateur 12
3.5 Exercice 5 : Authentification 13
3.6 Exercice 6 : Gestion de stock 14
3.7 Exercice 7 : Téléchargement 15
3.8 Exercice 8 : Validation de formulaire 15
3.9 Exercice 9 : Paiement 16
3.10 Exercice 10 : Conception 17

4 Partie 3 : Généricité **18**
4.1 Exercice 1 : 18
4.2 Exercice 2 : 19
4.3 Exercice 3 : 19
4.4 Exercice 4 : 20
4.5 Exercice 5 : 21
4.6 Exercice 6 : 22
4.7 Exercice 7 : 23
4.8 Exercice 8 : 24
4.9 Exercice 9 : 25
4.10 Exercice 10 : 25
4.11 Exercice 11 : 26
4.12 Exercice 12 : 27

5 Conclusion **28**

1 Introduction

Ce TP a pour objectif de maîtriser les concepts de la gestion des exceptions ainsi que la généricité en Java. À travers plusieurs exercices pratiques, nous avons appris à :

— gérer les exceptions système ;
— créer des exceptions personnalisées ;
— utiliser les classes et méthodes génériques ;
— améliorer la robustesse des applications Java.

2 Partie 1 : Exceptions système

2.1 Exercice 1 : Division sécurisée

Code Java :

```java
package partie1;
import java.util.Scanner;

public class Exercice1 {
    static class Calcul{
        public static int division(int a, int b) {
            if(b==0) {
                throw new ArithmeticException("Division par zéro interdite");
            }
            return a/b;
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.println("Entrer a:");
            int a = sc.nextInt();

            System.out.println("Entrer b:");
            int b = sc.nextInt();

            int resultat= Calcul.division(a, b);
            System.out.println("Résultat: " + resultat);

        } catch (InputMismatchException e) {
            System.out.println("Erreur: faut entrer des entiers correctes");
        } catch (Exception e) {
        System.out.println("Erreur de saisie !");
        }
    }
}
```

Résultat d'exécution :

```
Entrer a:
7
Entrer b:
0
Erreur de saisie !
```

```
Entrer a:
9
Entrer b:
3
Résultat: 3
```

2.2 Exercice 2 : NullPointerException

Code Java :

```java
package partie1;

public class Exercice2 {
    static class Calcul {
        public static void afficherLongueur(String s) {
            if (s == null) {
                System.out.println("la chaine est nulle");
            } else {
                System.out.println("la longueur: " + s.length());
            }
        }
        public static void afficherLongueurAvecException(String s) {
            try {
                System.out.println("la longueur: " + s.length());
            } catch (NullPointerException e) {
                System.out.println("Erreur: la chaîne nulle");
            }
        }
    }
    public static void main(String[] args) {
        String texte1="bonne journee";
        String texte2=null;

        System.out.println("Sans exception on a:");
        Calcul.afficherLongueur(texte1);
        Calcul.afficherLongueur(texte2);

        System.out.println("Avec try/catch on a:");
        Calcul.afficherLongueurAvecException(texte1);
        Calcul.afficherLongueurAvecException(texte2);
    }
}

```

Résultat d'exécution :

```
Sans exception on a:
la longueur: 13
la chaine est nulle
Avec try/catch on a:
la longueur: 13
Erreur: la chaîne nulle
```

2.3 Exercice 3 : Tableau sécurisé

Code Java :

```java
package partie1;
import java.util.Scanner;

public class Exercice3 {
    static class Tableau{
        public static void accesAvecIf(int[] tab, int i) {
            if (i >= 0 && i < tab.length) {
                System.out.println("Valeur: " + tab[i]);
            } else {
                System.out.println("Index invalide");
            }
        }
        public static void accesAvecTry(int[] tab, int i) {
            try {
                System.out.println("Valeur: " + tab[i]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Erreur: index hors limites");
            }
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int[] tab= {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

        System.out.print("Entrer le i: ");
        int i = sc.nextInt();

        System.out.println("Avec If:");
        Tableau.accesAvecIf(tab, i);

        System.out.println("Avec Try/Catch:");
        Tableau.accesAvecTry(tab, i);
    }
}
```

Résultat d'exécution :

```
Entrer le i: 9
Avec If:
Valeur: 10
Avec Try/Catch:
Valeur: 10
```

```
Entrer le i: 12
Avec If:
Index invalide
Avec Try/Catch:
Erreur: index hors limites
```

2.4 Exercice 4 : Conversion d'entier

Code Java :

```java
import java.util.Scanner;

public class Exercice4 {
    static class Conversion{
        public static int convertir(String s) {
            try {
                return Integer.parseInt(s);
            } catch (NumberFormatException e) {
                System.out.println("Erreur: conversion impossible");
                return 0;
            }
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Entrer une chaîne: ");
        String input = sc.nextLine();

        int resultat= Conversion.convertir(input);
        System.out.println("Résultat: " + resultat);
    }
}
```

Résultat d'exécution :

```
Entrer une chaîne: 89
Résultat: 89
```

```
Entrer une chaîne: bonjour
Erreur: conversion impossible
Résultat: 0
```

2.5 Exercice 5 : Choix d'exception

Code Java :

```java
package partie1;
import java.util.Scanner;
public class Exercice5 {
    static class Calcul {
        public static int racineCarree(int x) {
            if (x<0) {
                throw new IllegalArgumentException("Impossible de calculer la racine d'un nombre negatif");
            }
            return (int) Math.sqrt(x);
        }
    }
    public static void main(String[] args) {
        Scanner sc= new Scanner(System.in);
        try {
            System.out.println("entreu un nombre");
            int x= sc.nextInt();

            int resultat = Calcul.racineCarree(x);
            System.out.println("le resultat de la racine carree est:" +resultat);
        }catch(IllegalArgumentException e){
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
entreu un nombre
9
le resultat de la racine carree est:3
```

```
entreu un nombre
-3
Impossible de calculer la racine d'un nombre negatif
```

2.6 Exercice 6 : IllegalStateException

Code Java :

```java
package partie1;

public class Exercice6 {
    static class Machine{
        String etat="OFF";
        public void demarrer() {
            if(etat.equals("ON")) {
                throw new IllegalStateException("la machine est deja demarree");
            }
            etat="ON";
            System.out.println("La machine est demarree");
        }
    }
    public static void main(String[] args) {
        Machine mach= new Machine();
        try {
            mach.demarrer();
            mach.demarrer();
        }catch(IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
La machine est demarree
la machine est deja demarree
```

2.7 Exercice 7 : Propagation

Code Java :

```java
package partie1;

public class Exercice7 {
    static class Calcul{
        public static void C() {
            throw new ArithmeticException("Erreur dans C");
        }
        public static void B() {
            C();
        }
        public static void A() {
            try {
                B();
            }catch(ArithmeticException e) {
                System.out.println(e.getMessage());
            }
        }
    }
    public static void main(String[] args) {
        Calcul.A();
    }
}
```

Résultat d'exécution :

```
Erreur dans C
```

2.8 Exercice 8 : Checked Exception

Code Java :

```java
package partie1;
import java.util.Scanner;

public class Exercice8 {
    static class Calcul{
        public static void verifierAge(int age) throws Exception {
            if (age < 18) {
                throw new Exception("Age inferieur a 18");
            }
            System.out.println("Age valide");
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.println("entrer votre age:");
            int age= sc.nextInt();

            Calcul.verifierAge(age);

        }catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
entrer votre age:
19
Age valide
```

```
entrer votre age:
17
Age inferieur a 18
```

2.9 Exercice 10 : Comparaison

Code Java :

```java
package partie1;

public class Exercice10 {
    static class CompException {
        public static void verifierAge(int age) throws Exception {
            if (age < 18) {
                throw new Exception("Age inferieur a 18");
            }
            System.out.println("Age valide");
        }
    }
    static class CompRuntimeException{
        public static void verifierAge(int age) {
            if (age < 18) {
                throw new RuntimeException("Age inferieur a 18");
            }
            System.out.println("Age valide");
        }
    }
    public static void main(String[] args){
        try {
            CompException.verifierAge(11);
        }catch(Exception e) {
            System.out.println(e.getMessage());
        }

        try {
            CompRuntimeException.verifierAge(8);
        }catch(RuntimeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
Age inferieur a 18
Age inferieur a 18
```

==========

3 Partie 2 : Exceptions personnalisées

3.1 Exercice 1 : Compte bancaire

Code Java :

```java
package partie2;
class SoldeInsuffisantException extends Exception {
    public SoldeInsuffisantException(String msg) {
        super(msg);
    }
}
public class Exercice1 {
    static class CompteBancaire {
        int code;
        double solde;
        public CompteBancaire(int code, double solde) {
            this.code = code;
            this.solde = solde;
        }
        public void retirer(double montant) throws SoldeInsuffisantException {
            if (montant > solde) { throw new SoldeInsuffisantException("Solde insuffisant");}
            solde -= montant;
            System.out.println("le solde apres retrait:"+solde);
        }
        public void verser(double montant) {
            solde += montant;
            System.out.println("le solde apres versement:"+solde);
        }
    }
    public static void main(String[] args) {

        CompteBancaire c = new CompteBancaire(1, 90000);
        try {
            c.retirer(5600);
            c.verser(7000);

        } catch (SoldeInsuffisantException e) {
            System.out.println(e.getMessage());
        }
    }
```

Résultat d'exécution :

```
<terminated> Exercice1 (1) [Java Application] C:\
le solde apres retrait:84400.0
le solde apres versement:91400.0
```

3.2 Exercice 2 : Montant invalide

Code Java :

```java
package partie2;

class MontantInvalideException extends Exception {
    public MontantInvalideException(String message) {
        super(message);
    }
}
public class Exercice2 {
    static class Compte {
        public void retirer(double montant) throws MontantInvalideException {
            if (montant <= 0) {
                throw new MontantInvalideException("Montant invalide");
            }
            System.out.println("Retrait acceptee");
        }
    }
    public static void main(String[] args) {
        Compte c = new Compte();
        try {
            c.retirer(-10);
        } catch (MontantInvalideException e) {

            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
Montant invalide
```

3.3 Exercice 3 :Double validation

Code Java :

```java
package partie2;
class SoldeInsuffisantException extends Exception {
    public SoldeInsuffisantException(String message) {
        super(message);
    }
}
class MontantInvalideException extends Exception {
    public MontantInvalideException(String message) {super(message);}
}
public class Exercice3 {
    static class Compte {
        double solde;
        public Compte(double solde) {
            this.solde=solde;
        }
        public void retirer(double montant) throws SoldeInsuffisantException,MontantInvalideException {
            if (montant <= 0) {
                throw new MontantInvalideException(
                        "Montant invalide");
            }
            if (montant > solde) {
                throw new SoldeInsuffisantException(
                        "Solde insuffisant");
            }
            solde -= montant;
            System.out.println("le solde apres retrait:"+solde);
        }
    }
    public static void main(String[] args) {
        Compte c = new Compte(5000);
        try {
            c.retirer(-700);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
```

Résultat d'exécution :

```
<terminated> Exercice3 (1)
Montant invalide
```

3.4 Exercice 4 : Inscription Utilisateur

Code Java :

```java
package partie2;

class EmailInvalideException extends Exception {
    public EmailInvalideException(String message) {
        super(message);
    }
}
class AgeInvalideException extends Exception {
    public AgeInvalideException(String message) {
        super(message);
    }
}
public class Exercice4 {
    static class Utilisateur {
        public void inscrire(String email, int age)throws EmailInvalideException,AgeInvalideException {
            if (!email.contains("@")) {
                throw new EmailInvalideException("Email invalide");
            }
            if (age < 18) {
                throw new AgeInvalideException("Age invalide");
            }
            System.out.println("Inscription reussie");
        }
    }
    public static void main(String[] args) {
        Utilisateur u = new Utilisateur();
        try {
            u.inscrire("testgmail.com", 15);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice4 (
Email invalide
```

L'affichage quand on ecrit l'email correctement (test@gmail.com) :

```
<terminated> Exercice4
Age invalide
```

3.5 Exercice 5 : Authentification

Code Java :

```java
package partie2;

class AuthentificationException extends Exception {
    public AuthentificationException(String message) {
        super(message);
    }
}
public class Exercice5 {
    static class Login {
        public void login(String username,String password) throws AuthentificationException {
            if (!username.equals("admin") || !password.equals("1234")) {
                throw new AuthentificationException("Identifiants incorrects");
            }
            System.out.println("Connexion réussie");
        }
    }
    public static void main(String[] args) {
        Login l = new Login();
        try {
            l.login("user", "0000");
        } catch (AuthentificationException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
Identifiants incorrects
```

3.6 Exercice 6 : Gestion de stock

Code Java :

```java
package partie2;

class StockInsuffisantException extends Exception {
    public StockInsuffisantException(String message) {
        super(message);
    }
}
public class Exercice6 {
    static class Produit {
        int stock = 19;
        public void retirerDuStock(int quantite)throws StockInsuffisantException {
            if (quantite > stock) {
                throw new StockInsuffisantException("Stock insuffisant");
            }
            stock -= quantite;
            System.out.println("Retrait effectue");
        }
    }
    public static void main(String[] args) {
        Produit p = new Produit();
        try {
            p.retirerDuStock(40);
        } catch (StockInsuffisantException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice6 (1)
Stock insuffisant
```

3.7 Exercice 7 : Téléchargement

Code Java :

```java
package partie2;

class QuotaDepasseException extends Exception {
    public QuotaDepasseException(String message) {
        super(message);
    }
}
public class Exercice7 {
    static class Telechargement {
        public void telechargerFichier(double taille) throws QuotaDepasseException {
            double limite = 100;
            if (taille > limite) {
                throw new QuotaDepasseException("Quota depasse");
            }
            System.out.println("Téléchargement autorisé");
        }
    }
    public static void main(String[] args) {
        Telechargement t = new Telechargement();
        try {
            t.telechargerFichier(200);
        } catch (QuotaDepasseException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice7
Quota depasse
```

3.8 Exercice 8 : Validation de formulaire

Code Java :

```java
package partie2;

class ChampObligatoireException extends Exception {
    public ChampObligatoireException(String message) {
        super(message);
    }
}
public class Exercice8 {
    static class Formulaire {
        public void validerFormulaire(String nom, String email)throws ChampObligatoireException {
            if (nom.isEmpty()|| email.isEmpty()) {
                throw new ChampObligatoireException("Champ obligatoire vide");
            }
            System.out.println("Formulaire valide");
        }
    }
    public static void main(String[] args) {
        Formulaire f = new Formulaire();
        try {
            f.validerFormulaire("" , "");
        } catch (ChampObligatoireException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice8 (1) [Java Ap
Champ obligatoire vide
```

3.9 Exercice 9 : Paiement

Code Java :

```
<terminated> Exercice
Carte expirée
```

Résultat d'exécution :

```java
package partie2;
class PaiementRefuseException extends Exception {
    public PaiementRefuseException(String message) {
        super(message);
    }
}

class CarteExpireeException extends Exception {
    public CarteExpireeException(String message) {
        super(message);
    }
}
public class Exercice9 {
    static class Paiement {
        boolean carteExpiree = true;
        public void payer(double montant)throws PaiementRefuseException,CarteExpireeException {
            if (montant > 5000) {
                throw new PaiementRefuseException("Paiement refuse");
            }
            if (carteExpiree) {
                throw new CarteExpireeException("Carte expirée");
            }
            System.out.println("Paiement accepte");
        }
    }
    public static void main(String[] args) {
        Paiement p = new Paiement();
        try {
            p.payer(1000);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

3.10 Exercice 10 : Conception

1. Quand utiliser les checked exceptions ?

Les checked exceptions sont utilisées lorsque l'erreur peut arriver pendant l'exécution du programme et qu'elle doit être obligatoirement traitée par le programmeur. Java force la gestion de ces exceptions avec `try/catch` ou `throws`.

Par exemple :

— lecture d'un fichier ;

— connexion à une base de données ;

— accès à un serveur ou au réseau.

Elles permettent d'éviter l'oubli de certaines erreurs importantes.

2. Quand utiliser les unchecked exceptions ?

Les unchecked exceptions sont utilisées pour les erreurs liées à la logique du programme ou à une mauvaise utilisation du code. Le compilateur ne force pas leur gestion.

Par exemple :

— division par zéro ;

— accès à un index inexistant ;

— utilisation d'une valeur `null`.

Ces erreurs peuvent généralement être évitées en écrivant correctement le programme.

3. Pourquoi créer une exception personnalisée ?

Créer une exception personnalisée permet de rendre le programme plus clair et plus compréhensible. Au lieu d'utiliser uniquement les exceptions générales de Java, on peut créer des exceptions adaptées au besoin de l'application.

Par exemple :

— `SoldeInsuffisantException` ;

— `EmailInvalideException`.

Cela aide à afficher des messages plus précis et facilite la maintenance du code.

4 Partie 3 : Généricité

4.1 Exercice 1 :

Code Java :

```java
package partie3;

public class Exercice1 {
    static class Boite<T> {
        T contenu;
        public void setContenu(T contenu) {
            this.contenu = contenu;
        }

        public T getContenu() {
            return contenu;
        }
    }

    public static void main(String[] args) {

        Boite<String> b1 = new Boite<>();
        b1.setContenu("Hello");

        System.out.println(b1.getContenu());

        Boite<Integer> b2 = new Boite<>();
        b2.setContenu(15);

        System.out.println(b2.getContenu());
    }
}
```

Résultat d'exécution :

```
Hello
15
```

4.2 Exercice 2 :

Code Java :

```java
package partie3;

public class Exercice2 {
    static class Paire<T, U> {
        T premier;
        U second;
        public Paire(T premier, U second) {
            this.premier = premier;
            this.second = second;
        }

        public void afficherPaire() {
            System.out.println("Premier : " + premier);
            System.out.println("Second : " + second);
        }
    }

    public static void main(String[] args) {

        Paire<String, Integer> p =new Paire<>("Age", 20);
        p.afficherPaire();
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice2 (2) [Java A
Premier : Age
Second : 20
```

4.3 Exercice 3 :

Code Java :

```java
package partie3;
interface Calcul<T> {
    T addition(T a, T b);
}
public class Exercice3 {
    static class CalculInteger implements Calcul<Integer> {
        public Integer addition(Integer a, Integer b) {
            return a + b;
        }
    }
    static class CalculDouble implements Calcul<Double> {
        public Double addition(Double a,Double b) {
            return a + b;
        }
    }
    public static void main(String[] args) {
        CalculInteger c1 =new CalculInteger();
        System.out.println(c1.addition(4, 5));
        CalculDouble c2 =new CalculDouble();
        System.out.println(c2.addition(7.8, 10.9));
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice3
9
18.7
```

4.4 Exercice 4 :

Code Java :

```java
package partie3;

interface Comparateur<T> {
    int comparer(T a, T b);
}
public class Exercice4 {
    static class ComparateurInteger implements Comparateur<Integer> {
        public int comparer(Integer a, Integer b) {
            return a.compareTo(b);
        }
    }
    static class ComparateurString implements Comparateur<String> {
        public int comparer(String a,String b) {
            return Integer.compare(
                    a.length(),
                    b.length());
        }
    }
    public static void main(String[] args) {
        ComparateurInteger c1 = new ComparateurInteger();
        System.out.println(c1.comparer(17, 12));
        ComparateurString c2 = new ComparateurString();
        System.out.println(c2.comparer("Hello","Bonjour"));
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice4 (
1
-1
F
```

4.5 Exercice 5 :

Code Java :

```java
package partie3;

public class Exercice5 {
    static class Boite<T> {
        T valeur;
        public Boite(T valeur) {
            this.valeur = valeur;
        }
        public void afficher() {
            System.out.println("Valeur : " + valeur);
        }
    }
    public static void main(String[] args) {
        Boite<String> b1 =new Boite<>("Hello");
        b1.afficher();
        Boite<Integer> b2 =new Boite<>(90);
        b2.afficher();
    }
}
```

Résultat d'exécution :

```
Valeur : Hello
Valeur : 90
```

4.6 Exercice 6 :

Code Java :

```java
package partie3;

public class Exercice6 {
    public static <T> void afficherTableau(T[] tableau) {
        for (T element : tableau) {
            System.out.println(element);
        }
    }
    public static <T> T getPremier(T[] tableau) {
        return tableau[0];
    }
    public static void main(String[] args) {
        Integer[] tab = {0, 4, 3, 9};
        afficherTableau(tab);
        System.out.println("Premier : "+ getPremier(tab));
    }
}
```

Résultat d'exécution :

```
0
4
3
9
Premier : 0
```

4.7 Exercice 7 :

Code Java :

```java
package partie3;

public class Exercice7 {
    public static <T extends Number>
    double somme(T a, T b) {
        return a.doubleValue() + b.doubleValue();
    }
    public static void main(String[] args) {
        System.out.println(somme(39, 22));
        System.out.println(somme(4.5, 6.7));
    }
}
```

Résultat d'exécution :

```
61.0
11.2
```

4.8 Exercice 8 :

Code Java :

```java
package partie3;

class Animal<T> {
    T nom;
    public Animal(T nom) {
        this.nom = nom;
    }

    public void afficher() {
        System.out.println("Nom : " + nom);
    }
}
class Chien extends Animal<String> {
    public Chien(String nom) {
        super(nom);
    }
}
public class Exercice8 {
    public static void main(String[] args) {
        Chien c = new Chien("Bella");
        c.afficher();
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice8
Nom : Bella
```

4.9 Exercice 9 :

Code Java :

```java
package partie3;

class Vehicule<T> {
    T vitesse;
    public Vehicule(T vitesse) {
        this.vitesse = vitesse;
    }

    public void afficher() {
        System.out.println("Vitesse : " + vitesse);
    }
}
class Voiture<T> extends Vehicule<T> {
    public Voiture(T vitesse) {
        super(vitesse);
    }
}
public class Exercice9 {
    public static void main(String[] args) {
        Voiture<Integer> v =new Voiture<>(180);
        v.afficher();
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice9 (1) [
Vitesse : 180
```

4.10 Exercice 10 :

Code Java :

```java
package partie3;

class Repository<T> {
    public void save(T obj) {
        System.out.println("Objet sauvegardé : "+ obj);
    }
}

class User {
    String nom;
    public User(String nom) {
        this.nom = nom;
    }
    public String toString() {
        return nom;
    }
}
class UserRepository extends Repository<User> {
}
public class Exercice10 {
    public static void main(String[] args) {
        User user = new User("Hidaya");
        UserRepository ur = new UserRepository();
        ur.save(user);
    }
}
```

Résultat d'exécution :

```
<terminated> Exercice10 (1) [Java Applic
Objet sauvegardé : Hidaya
```

4.11 Exercice 11 :

Code Java :

```java
package partie3;
import java.util.List;
import java.util.ArrayList;

public class Exercice11 {
    public static void afficherListe(List<?> liste) {
        for (Object obj : liste) {
            System.out.println(obj);
        }
    }
    public static void main(String[] args) {
        List<String> liste = new ArrayList<>();
        liste.add("Java");
        liste.add("C");
        afficherListe(liste);
    }
}
```

Résultat d'exécution :

```
Java
C
```

4.12 Exercice 12 :

Code Java :

```java
package partie3;
import java.util.List;
import java.util.ArrayList;

public class Exercice12 {
    public static void afficherNombres(List<? extends Number> liste) {
        for (Number n : liste) {
            System.out.println(n);
        }
    }
    public static void main(String[] args) {
        List<Integer> l1 = new ArrayList<>();

        l1.add(9);
        l1.add(15);

        List<Double> l2 = new ArrayList<>();

        l2.add(1.2);
        l2.add(3.4);

        afficherNombres(l1);
        afficherNombres(l2);
    }
}
```

Résultat d'exécution :

```
9
15
1.2
3.4
```

5 Conclusion

Ce TP nous a permis de comprendre l'importance :

— de la gestion des exceptions pour sécuriser les programmes ;

— des exceptions personnalisées pour améliorer la clarté du code ;

— de la généricité pour rendre les classes et méthodes réutilisables.

Grâce à ces exercices, nous avons renforcé nos compétences en programmation orientée objet avec Java.